

RegRadar Security and Access Document

Contents

RegRadar — Security & Access Document	1
1. How to Read This Document	1
2. Authentication: How RegRadar Verifies Who's Calling	1
3. User Roles & Permissions	3
4. Row-Level Security: How the Database Protects Data on Its Own	5
5. Error Handling Guide: What Happens When Things Go Wrong	6
6. Edge Cases to Handle Before Launch	11
7. Pre-Launch Security Checklist	12

RegRadar — Security & Access Document

Version 1.0 | Written for a non-technical founder audience

This document explains, in plain English, how RegRadar decides who is allowed to do what, how the database protects data even if something upstream goes wrong, what happens when any part of the system fails, and what sharp edges need to be handled before you let real customers in. Technical terms are explained the first time they appear.

1. How to Read This Document

RegRadar has no login screen in its first version — it's a system that other software (and a Slack channel and an inbox) talk to, not something a person clicks into. That single fact shapes almost every security decision below: there's no "user sitting at a keyboard" to protect with a password, but there are several kinds of *credentials* — API keys, webhook secrets, service accounts — that all need the same care a password would get, because each one is a way into your customer's regulatory data.

Think of this document as answering five questions a careful founder should be able to answer before taking money for this product: Who can prove they're allowed in? What exactly can each kind of visitor do once they're in? What stops the database itself from leaking data even if the application code has a bug? What happens the moment any single piece breaks? And what weird situations have we thought through in advance instead of discovering them in production?

2. Authentication: How RegRadar Verifies Who's Calling

Recommendation: API key authentication for V1, with a clear upgrade path to full login (SSO) when a dashboard ships in V2.

Why API keys, not a login page

RegRadar's product isn't a website someone logs into — it's a REST API (a set of web addresses that other programs call to get data) plus alerts that show up in Slack and email. The people who actually

touch RegRadar directly are the Engineering Lead wiring it into internal tools, and automated systems. Building a full login system (username/password, “forgot password” flows, session management) for a product nobody logs into would be solving a problem you don’t have yet, and every login system you build is also a login system you have to defend against attackers — so V1 deliberately skips it.

Instead, each customer organization is issued one or more **API keys** — long, random secret strings, similar to how Stripe or Twilio issue keys. A key is included in every request in a header (a piece of metadata sent with the request, not visible in the address bar or browser history) called *Authorization*. If the key is missing, wrong, or has been revoked, the request is rejected before it ever touches customer data.

How keys are handled safely

A few rules make this safe rather than just simple:

Keys are never stored in a readable form. The database stores a one-way scrambled version of each key (a *hash*), the same way a password would be stored. Even someone with full database access can’t read out a working key — only RegRadar can verify that a key you present matches the hash on file, the same way a lock recognizes the right key without “knowing” what the key looks like in advance.

Keys are shown exactly once. When a new key is created, the real value is displayed one time, at creation, and never again. If it’s lost, the only fix is issuing a new one — this is standard practice and prevents keys from being casually retrievable later by anyone who gains access to an admin screen.

Keys can be rotated without downtime. A customer (or you) can generate a new key while the old one still works, switch systems over, and only then revoke the old one — so rotating a leaked key doesn’t mean an outage.

Keys are scoped, not all-powerful. Every key is tied to a role (see Section 3) that limits exactly what it can do. A key handed to a Slack integration doesn’t need — and won’t have — the ability to change which regulators RegRadar monitors.

Every request is logged with which key made it. If something looks wrong later, you can trace every action back to a specific key and, from there, to the team member or system responsible for it.

Keys only work over encrypted connections (HTTPS). RegRadar should refuse any request that isn’t encrypted in transit, so a key can never be intercepted by someone else on the same network.

Outbound trust: webhook signing

RegRadar also sends data *out* — to Slack, to email, and to webhook URLs that customers register (a webhook is just a URL that RegRadar sends a POST request to whenever something new happens, so the customer’s own systems can react automatically). Anyone could, in theory, stand up a fake server and claim to be RegRadar. To prevent that, every outbound webhook is **signed** with a secret unique to that one webhook registration: RegRadar includes a cryptographic stamp (an HMAC signature) that only someone who knows the shared secret could have produced. The receiving system checks the stamp before trusting the payload. This is the same technique Stripe and GitHub use for their webhooks.

Internal system credentials

The pipeline itself (the automated agents that ingest filings, classify them, and generate briefs) needs to read and write data too, but these are not customer-facing keys — they’re internal service credentials that live only inside RegRadar’s private infrastructure and are never exposed to the public internet or returned in any API response. This distinction matters: a leaked customer API key is bad, but a leaked internal service credential is catastrophic, because it can bypass the row-level protections described in Section 4. These credentials should live in a secrets manager (not a `.env` file committed to code) and should be rotated on a schedule even if nothing has gone wrong.

What's deliberately deferred to V2

Once a dashboard exists and real humans log in with a username and a “sign in” button, that's the moment to add proper **single sign-on (SSO)** — letting a customer's compliance team log in with the Google or Microsoft account their company already manages, rather than RegRadar inventing its own password system. This also enables multi-factor authentication (MFA) for free, since most SSO providers already require it. Building this now would be premature; the API-key model and a future SSO-based login can coexist without conflict, because a person logging in through SSO would simply be issued an API key behind the scenes.

One thing to lock down immediately, even before writing code

The handful of places *you* directly touch secrets — your cloud hosting dashboard, your database provider's dashboard, your GitHub account — should have MFA turned on today. Nearly every real-world breach of an early-stage product starts with one of these accounts, not with the product's own authentication logic.

3. User Roles & Permissions

Because there's no login screen, a “role” in RegRadar is really a **type of API key**. Each key is issued with exactly one role, and that role determines what it can see and do. Below is every role, described in plain terms, followed by a full permission matrix.

3.1 The roles, in plain English

Admin — Usually held by the founder or a designated technical lead at the customer's company. This is the “master key” role: it can do everything a normal role can, plus manage other keys (create, view, and revoke them) and change which regulators and topics RegRadar is watching. Because this role is the most powerful, it should be issued to as few people as possible, and every action it takes should be logged.

Compliance Analyst — The person who actually reads filings in detail every day. Needs full access to the extracted detail (obligations, deadlines, risk flags) so they don't have to read the original 80-page document, and needs to be able to search past filings. Does not need to touch configuration or manage other people's integrations.

Executive / Chief Compliance Officer (read-only, high-level) — Wants a short brief and a risk score, not the underlying detail. This role is deliberately given *less* access than the Analyst role — not because the CCO is less trusted, but because this key is more likely to end up embedded in a Slack app or forwarded around, so it's built to be low-risk if it ever leaks: it can read summaries, not the full structured extraction or raw documents.

Legal Counsel — Needs to search the full historical archive of filings to answer “what has the SEC said about X before.” Read-only, but broader than the Executive role because deep search is the whole point of this persona's access.

Engineering Lead / Integrator — The person wiring RegRadar into the company's other internal tools. Needs full read access plus the ability to register and manage webhooks, and needs visibility into cost/performance metrics to forecast spend. Does not need to change what RegRadar monitors — that's a business decision, not an engineering one, and is reserved for Admin.

System / Service Roles (not human) — The Ingestion Agent, the pipeline workers, and the Delivery Agent each have their own narrow internal credential. These are described separately at the end of this section because they don't behave like customer-facing roles at all — they run entirely inside RegRadar's private infrastructure.

Unauthenticated visitor — Anyone with no key at all. Can only reach a basic health-check endpoint that says “the service is up,” nothing else.

3.2 Full permission matrix

Action	Admin	Analyst	Executive (CCO)	Legal Counsel	Eng Lead
View list of filings	Yes	Yes	Yes (High/Critical only, brief-level)	Yes	Yes
View full filing detail (obligations, deadlines, risk flags)	Yes	Yes	No	Yes	Yes
View persona-tailored brief	Yes	Yes	Yes (own brief only)	Yes	Yes (own brief only)
Run natural-language search (RAG) across all filings	Yes	Yes	No	Yes	Yes
Register a new webhook	Yes	No	No	No	Yes
View / delete a webhook	Yes (any)	No	No	No	Yes (own only)
View cost & performance metrics	Yes	No	No	No	Yes
Change which regulators/domains are monitored	Yes	No	No	No	No
Create or revoke other API keys	Yes	No	No	No	No
View raw source PDFs	Yes	Yes	No	Yes	No (not needed for integration work)

A few things worth calling out explicitly, in plain English:

No customer-facing role — not even Admin — can directly write to the filings, extractions, or briefs data. Only the internal system agents can create or edit that data. This is a deliberate choice: if someone's API key is ever stolen, the attacker can *read* whatever that role is allowed to read, but they can't plant a fake filing, alter a risk score, or forge a brief. Read access leaking is bad; write access leaking is far worse, so write access to core data isn't given to any human-facing key at all.

The Executive role is intentionally the most restricted "real" role, not the most trusted. This sometimes surprises founders — shouldn't the CCO have the most access? But security roles are sized to what a job actually requires plus how exposed that credential is likely to be, not to seniority. A CCO's key is likely to live inside a Slack app config touched by fewer technical safeguards than an engineer's; giving it narrower scope limits the damage if it's ever exposed.

3.3 Internal system roles (not customer-facing)

Role	What it can do	Why it's separate
Ingestion Agent	Write new filing records, update polling state	Only role allowed to create new filings — this is what makes “no customer key can write filings” enforceable
Pipeline Worker (classification, retrieval, analysis, summarization)	Write chunks, extractions, and briefs for filings already ingested	Runs entirely inside the private network; never accepts a request from the public internet
Delivery Agent	Read briefs, write delivery records, read webhook signing secrets to sign outgoing payloads	The <i>only</i> role ever allowed to read a webhook's secret in plain form — every other role, including Admin, only ever sees that a secret exists, never its value, after creation

4. Row-Level Security: How the Database Protects Data on Its Own

Row-level security (RLS) is a feature of the database itself (Postgres, which is what powers Supabase, RegRadar's database) that decides, row by row, whether a given request is even allowed to see or touch that row — enforced by the database, not by application code. The reason this matters to a founder: application code has bugs. An engineer can forget to add a permission check in one new API endpoint. RLS is a second, independent layer that keeps working even if that mistake happens, because the database itself refuses to hand back rows the requester isn't allowed to see, no matter what the application code asked for.

An important distinction, explained plainly

Row-level security controls **which rows** you can see — for example, “only see filings belonging to your organization,” or “only see webhooks you personally registered.” It does **not** control **which fields within a row** you can see. Hiding the raw obligations data from the Executive role while still letting them see the brief for that same filing is handled by the *application layer* (the API code itself deciding what to include in its response), not by the database. Both layers are necessary and neither substitutes for the other — this document covers both so nothing falls through the gap between them.

4.1 The baseline rule

Every table containing customer data has RLS turned on, with **no default access** — meaning that unless a specific rule explicitly allows a request, it's denied. This “deny by default” posture is the single most important security decision in this whole document: it means a newly added table is safe by default, and a forgotten policy fails closed (denies access) rather than failing open (accidentally allowing access).

The one exception is the internal service credential used by the pipeline agents (Section 3.3), which is intentionally allowed to bypass row-level security — because it has to write across the whole system to do its job. This is exactly why that credential must never be exposed outside RegRadar's private infrastructure: it is the one key that ignores every rule below.

4.2 Rules per table

filings (the core record for each regulatory document) — Any valid customer key can read filings belonging to their own organization. No customer key can insert, edit, or delete a filing; only the Ingestion Agent can create one, and only the Pipeline Worker can update its processing status.

filing_chunks (the pieces of a document used for search) — Not exposed to any customer key directly at all. These rows only exist to power the search endpoint internally; a customer never queries this table directly, so there's no policy that grants row access to any human-facing role. This closes off an entire category of risk: even a fully compromised Analyst key can't page through raw chunk data to try to reconstruct a whole document more cheaply than intended.

extractions (the structured obligations/deadlines/risk data) — Readable by Admin, Analyst, Legal Counsel, and Eng Lead, scoped to their own organization. Not readable by the Executive role at the database level either — reinforcing, at the data layer, the same restriction described in Section 3.

briefs (the plain-English summaries) — Readable by all roles within the organization; this is the one table every role has full read access to, since it's designed to be the safe, already-distilled version of the data.

webhooks — A key can only see and manage the webhook rows *it personally created*, with one exception: Admin can see and manage every webhook registered under the organization. This prevents one Engineering Lead's key from silently disabling or redirecting an integration another team member built.

deliveries (the log of every alert sent) — Readable within the organization, but the webhook signing secret is never included in what gets returned, at either the database or application layer, to anyone — including Admin — after the moment of creation.

source_configs (which regulators/domains are being watched) — Readable by everyone in the organization (so any role can see what's being monitored), but only editable by Admin.

eval_runs (quality/cost metrics) — Readable only by Admin and Eng Lead. This is treated as sensitive business information (it reveals what RegRadar's per-document cost is, which is competitively and financially sensitive), not as compliance information, so it's scoped separately from the compliance-facing roles.

api_keys — A key can see its own metadata (when it was created, when it was last used) but never its own or anyone else's actual secret value, which is never stored in a readable form in the first place (Section 2). Admin can see metadata for every key in the organization, but still never the actual secret.

4.3 Building in multi-organization safety now, even as a single-customer product

RegRadar's V1 is built for one compliance team at a time, but nearly every early-stage product ends up onboarding a second customer sooner than planned, and retrofitting organization boundaries into a database that was never designed for them is a genuinely risky migration to do under time pressure. The recommendation here is to add an `organization_id` column to every customer-data table **from day one**, even while there's only one organization in the system, and to write every row-level security rule in terms of "same organization as the requester" rather than "the only organization that exists." This costs almost nothing to do now and removes an entire class of future risk.

4.4 Before launch: prove the rules actually work

A row-level security rule that hasn't been tested is a row-level security rule you're hoping works. Before launch, there should be an automated test for every row listed above that specifically tries to access data it *shouldn't* be able to, using each role's credentials, and confirms the database says no. This is cheap to write once and should run automatically on every code change, the same way any other test would.

5. Error Handling Guide: What Happens When Things Go Wrong

Every row below follows the same pattern: what can go wrong, what RegRadar should do automatically without a person intervening, what the customer actually experiences, and what gets logged so an

engineer can investigate. The goal stated plainly: **a failure in one filing should never take down the whole system, and a customer should never see a raw technical error message.**

5.1 Ingestion (pulling filings from SEC, FDA, FINRA)

What Can Go Wrong	Automatic Response	What The Customer Sees	What Gets Logged
A regulator's feed is temporarily down or slow	Retry with increasing delays between attempts (so a brief outage doesn't cause a flood of failed requests); give up gracefully after a set number of tries and try again on the next scheduled check	Nothing — this is invisible unless the outage is long enough to delay an alert, in which case a filing simply arrives a few minutes later than usual	A warning noting the source and how long it's been unreachable
SEC EDGAR rate limit is hit (they allow a limited number of requests per second)	Automatically slow down to stay under the limit; never let the ingestion process burst past it	Nothing	A note that the rate limit was approached, for capacity planning
The same filing is detected twice (e.g., a retry after a network blip)	Recognize the duplicate using the regulator's own document ID before processing it a second time, and silently discard the repeat	Nothing — critically, the customer never gets the same alert twice	A note that a duplicate was caught, for monitoring purposes
A downloaded PDF is corrupted or incomplete	Discard it, log the failure, retry the download once, and flag for manual review if it fails again	Nothing, unless the filing never successfully downloads, in which case it simply never appears (see the edge-case note in Section 6 about monitoring for "missing" filings)	An error with the source URL and failure reason

5.2 Reading the document (parsing, chunking, and understanding structure)

What Can Go Wrong	Automatic Response	What The Customer Sees	What Gets Logged
The PDF is a scanned image with no extractable text	Detected automatically; the filing is flagged as needing OCR (optical character recognition — converting an image of text into actual readable text) rather than silently producing an empty or wrong summary	The filing shows a “processing — scanned document” status rather than a wrong or missing brief	A flag for the known-limitations list; this is explicitly out of scope for V1 per the product plan, so it should fail visibly, not silently
The document is unusually long (beyond what a single pass can process)	Split and processed in sections, then combined, rather than truncated	No visible difference in the final brief	Processing time and section count, for monitoring
A table or exhibit doesn’t parse cleanly	The malformed content is excluded from that chunk rather than allowed to corrupt the surrounding text	The extraction may simply omit that specific table’s data rather than showing garbled text	A flag noting which filing had a parsing issue, added to a running “hard cases” list used to improve chunking over time

5.3 Classification (deciding what kind of filing this is and how urgent it is)

What Can Go Wrong	Automatic Response	What The Customer Sees	What Gets Logged
The classification model is down or times out	Retry once; if it still fails, hold the filing in a “needs classification” state rather than guessing	A short delay before the alert arrives; no incorrect risk label is ever shown	An alert to the engineering team if this happens repeatedly, since it blocks the whole pipeline
The model is confident but wrong (misclassifies a genuinely urgent filing as low priority)	For filings above a certain risk threshold, a second, independent check (a spot-check by a more capable model) is run before the priority is finalized	Nothing different in the normal case; in the rare disagreement case, the more cautious of the two results wins	Both results are logged whenever they disagree, building a dataset to improve accuracy over time
The model is unsure (low confidence either way)	Default to treating it as higher priority rather than lower — it’s far better to over-alert occasionally than to silently miss something important	The filing may get flagged for human review before an alert goes out	The confidence score is recorded on every filing for later analysis

5.4 Search and retrieval (finding related past filings)

What Can Go Wrong	Automatic Response	What The Customer Sees	What Gets Logged
The database search takes too long	A timeout cuts the search short and returns the best results found so far rather than hanging indefinitely	A search that returns quickly with results, possibly slightly less complete, rather than a spinner that never resolves	Query time is tracked so slow searches can be investigated
No relevant past filings exist yet (e.g., very early in the product's life, before much history has built up)	The system proceeds without historical context rather than failing	The brief is generated using only the current document, and may note that no similar past filings were found	Nothing unusual — this is expected and normal early on
The AI model used to generate answers to search questions isn't available	Fall back to keyword-based search rather than failing the whole request	Search results are returned but the response notes that natural-language answering is temporarily limited	An alert if this fallback is being used frequently, since it signals a dependency outage

5.5 Analysis (extracting obligations, deadlines, and risk detail) and Summarization

What Can Go Wrong	Automatic Response	What The Customer Sees	What Gets Logged
The AI provider (OpenAI or the summarization model) is down or rate-limited	Automatically switch to the designated backup model for that tier of filing rather than stalling	The brief may be slightly less detailed than usual but still arrives on time	Every time a fallback model is used, so the team can see how often the primary model is unavailable
The AI returns output that doesn't match the expected structure (e.g., missing a required field)	Automatically retry the request once with a stricter prompt before giving up	If it still fails, the filing is marked as needing review rather than showing an incomplete or broken brief	The malformed response is saved for debugging, not discarded
A generated brief contains a claim that can't be traced back to the source document (a "hallucination" — the AI stating something as fact that isn't actually in the filing)	Automated quality checks run on a sample of briefs before and after any change to the system; if quality drops below the agreed threshold, that change is blocked from going live at all	In normal operation, nothing — this is a safeguard that runs before customers ever see the affected version	Every quality check result is recorded and tracked over time so a slow decline is caught, not just sudden failures
The day's AI spending approaches the budget limit	A spending circuit breaker automatically switches remaining filings that day to the cheaper backup model, and stops entirely if a hard ceiling is reached	Alerts may arrive with a slightly shorter delay margin, but processing does not stop entirely except in an extreme, clearly-alarmed scenario	A budget alert is sent to the team the moment the threshold is crossed, well before any hard stop

5.6 Delivery (Slack, email, and webhooks)

What Can Go Wrong	Automatic Response	What The Customer Sees	What Gets Logged
The Slack integration is disconnected or misconfigured	Automatically fall back to email so a Critical alert never simply disappears	An email arrives instead of a Slack message, possibly noting that Slack delivery failed	An alert to the account owner that Slack needs to be reconnected
A customer's registered webhook endpoint is down or returns an error	Retry with increasing delays a limited number of times; after repeated consistent failures, automatically pause that webhook rather than retrying forever	The customer's engineering team can see delivery failures listed against that webhook if they check its status	The webhook is flagged so the Engineering Lead is notified their endpoint needs attention
An email fails to send (bounced address, provider outage)	Retry once; if it still fails, the failure is recorded rather than silently dropped	The recipient doesn't receive the email; this becomes visible in a delivery log	An alert if email failures spike, since that usually means a provider-wide issue
The same alert is accidentally about to be sent twice	Checked against the delivery log before sending — if a delivery record already exists for that filing and channel, it's not sent again	The customer never receives a duplicate alert	Nothing unusual to log; this is the normal duplicate-prevention path working as intended

5.7 The API itself (what happens when someone calls RegRadar directly)

What Can Go Wrong	Automatic Response	What The Customer Sees	What Gets Logged
An API key is missing, wrong, or has been revoked	The request is rejected immediately, before touching any data	A clear "not authorized" message, never a confusing technical error	Every rejected attempt is logged; repeated failures from the same source may indicate someone guessing at keys
A key is calling far more often than its allowed rate	Extra requests are rejected with a clear "slow down" response and a note on when to try again	A clear rate-limit message rather than a slow or broken experience for everyone else sharing the system	Repeated rate-limit hits are tracked per key, both to protect the system and to flag customers who may need a higher limit
The database or an internal dependency is briefly unavailable	The request fails cleanly with a clear "temporarily unavailable, try again" response rather than hanging or returning wrong data	A short, clear error rather than a long wait or incorrect results	An alert to the engineering team the moment this happens

What Can Go Wrong	Automatic Response	What The Customer Sees	What Gets Logged
Someone sends a malformed or unexpected request	Rejected with a specific, plain description of what was wrong with the request	A clear error describing the problem, so the Engineering Lead persona can fix their integration quickly	Nothing alarming — this is normal, expected validation behavior

6. Edge Cases to Handle Before Launch

These are the situations that don't show up in a demo but will eventually show up in production — the kind of thing a security engineer flags now specifically so it isn't discovered the hard way later.

Someone tries to trick the AI through the document itself. RegRadar feeds the text of every filing into an AI model to extract obligations and generate summaries. In theory, a filing could contain text specifically crafted to manipulate that AI — for example, a hidden instruction telling it to mark a serious filing as low risk. This is a known category of attack called *prompt injection*. Before launch, the analysis process should treat all document text as data to be analyzed, never as instructions to be followed, and the automated quality checks (Section 5.5) should include test cases specifically designed to catch this.

A registered webhook URL could point somewhere it shouldn't. If a customer (or an attacker with a stolen key) registers a webhook pointing at an internal address rather than a real external server, RegRadar could be tricked into making requests to its own private infrastructure. Webhook URLs should be validated at registration time to block internal/private network addresses.

A webhook payload could be captured and replayed later. Even with signing (Section 2), a captured, validly-signed request could technically be resent later by an attacker who intercepted it. Signed payloads should include a timestamp, and the receiving check should reject anything outside a short window (a few minutes), so an old captured payload can't be replayed.

Two ingestion checks could pick up the same filing at almost the same moment, before the first one has finished recording it, risking two separate processing runs for the same document. There needs to be a hard, database-enforced rule that a given source document ID can only ever have one filing record, not just an application-level check that could race.

A customer's Slack workspace gets disconnected without anyone noticing. If nobody is watching for this, Critical alerts could silently stop arriving until someone happens to notice. There should be an active check that periodically confirms Slack delivery is actually working, not just a passive "we'll find out if someone complains."

A filing simply never shows up, because it failed silently somewhere in the pipeline. Since the customer doesn't know what they don't know, there should be a way to notice "we expected roughly this many filings from this source in this time window and got far fewer" rather than relying entirely on individual failure alerts.

Test activity could leak into a real customer's Slack channel or inbox. Before launch, test and production API keys (and their associated Slack/email destinations) must be completely separate, so a staging run never sends a real-looking alert to an actual compliance team.

A key rotation could momentarily lock a customer out. Rotating a key should support a brief overlap window where both the old and new key work, rather than the old key dying the instant the new one is created.

Runaway retries could quietly turn into a runaway bill. Every retry loop described in Section 5 needs an absolute cap on attempts. Without one, a persistent failure (say, a consistently-down

third-party API) could cause the system to hammer that dependency — and rack up AI provider costs — indefinitely.

Regulatory filings may contain personal information about individuals, not just companies (names in disclosures, complaints, or exhibits, for example). Before launch, decide explicitly how long raw PDFs and extracted data are retained, and make sure that decision is written down and defensible, not left implicit. This matters even though RegRadar isn't itself a legal advisory tool — it's still handling and storing potentially sensitive documents on a customer's behalf.

An entire AI provider could go down for an extended period, not just time out briefly. Beyond the per-request fallback already described, there should be a clear, tested answer to “what does RegRadar do if OpenAI is down for six hours” — even if the honest answer for V1 is “alerts run slower using only the backup model,” that should be a decision made in advance, not discovered live.

A single company's compliance team may eventually want more than one workspace (e.g., separate teams for different business units). Even though multi-organization support is explicitly out of scope for V1, Section 4.3's recommendation to add organization boundaries into the database now is what keeps this from becoming an emergency rebuild later.

7. Pre-Launch Security Checklist

A short, concrete list to work through before onboarding a real customer:

MFA is turned on for every account that can touch production secrets (hosting provider, database provider, code repository).

Every database table has row-level security turned on with a deny-by-default policy, and there's an automated test proving each role can only see what it's supposed to see.

API keys are hashed at rest, shown once at creation, individually revocable, and rate-limited per key.

Every outbound webhook payload is signed, includes a timestamp, and rejects replays outside a short time window.

Webhook URLs are validated at registration to block internal/private network addresses.

Every retry loop in the system has a maximum attempt count.

Test and production credentials (API keys, Slack destinations, email addresses) are fully separated.

There's an active, ongoing check confirming that Slack and email delivery are actually working, not just an assumption that they are.

A written data-retention decision exists for raw PDFs and extracted data, and it's actually implemented, not just documented.

The automated quality checks that gate every AI-prompt change include at least one test case specifically designed to catch prompt injection from document content.